

CLOUD NATIVE
COMPUTING FOUNDATION

Kelpflux

Elastic Slurm scheduling on Kubernetes
for shared GPU AI workloads.

Agenda

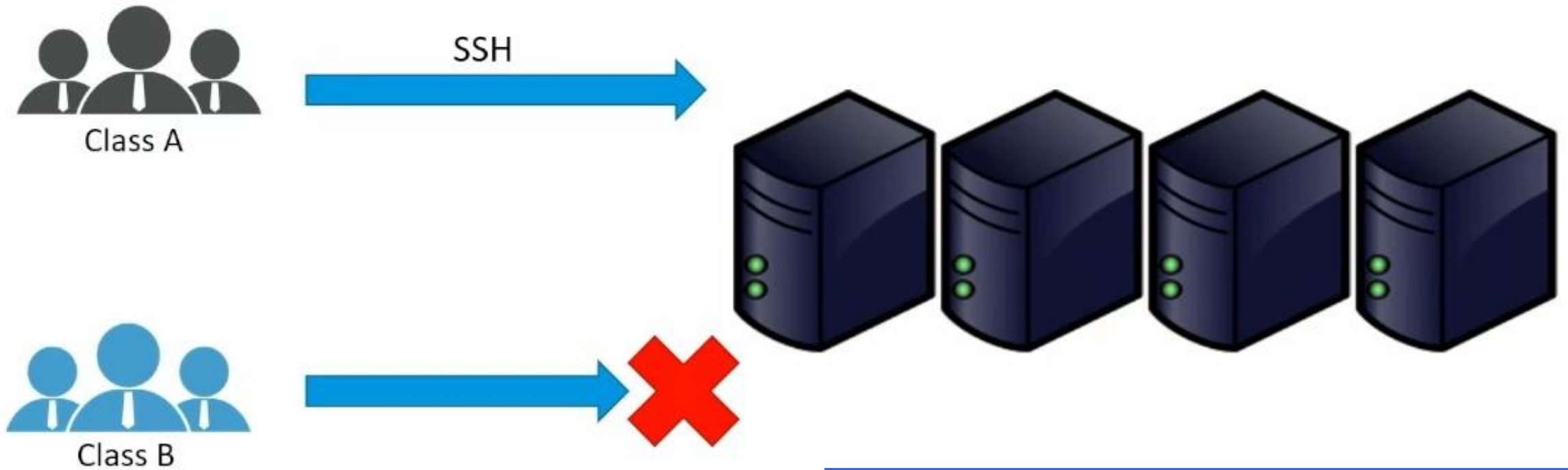
-  **1 Introduction**
-  **2 System Architecture**
-  **3 Demo & Evaluation**
-  **4 References**



01. Introduction

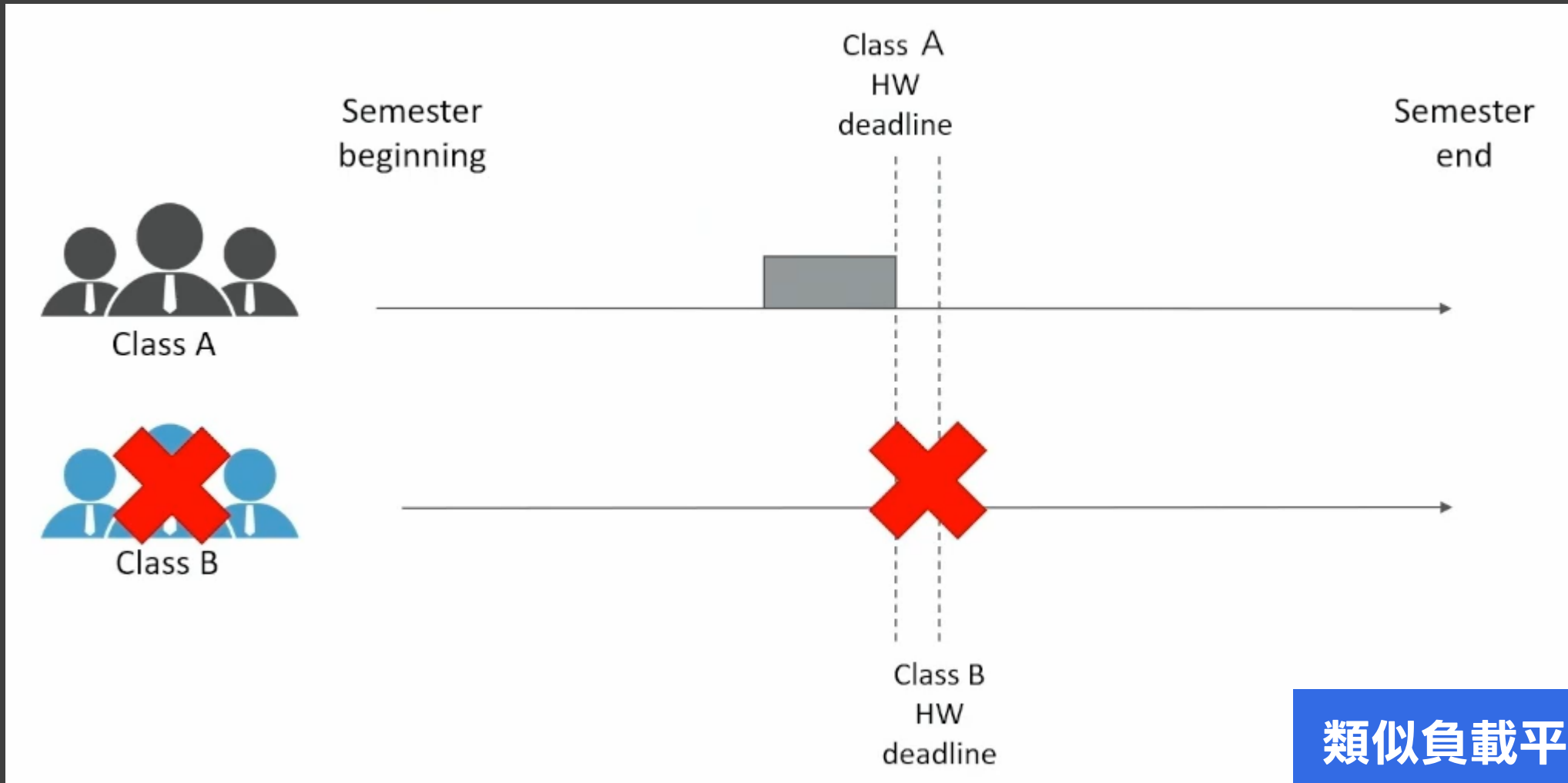
專題簡介

Example Scenarios



有沒有辦法多台機器給多個人使用？

Example Scenarios



類似負載平衡的概念

Slurm

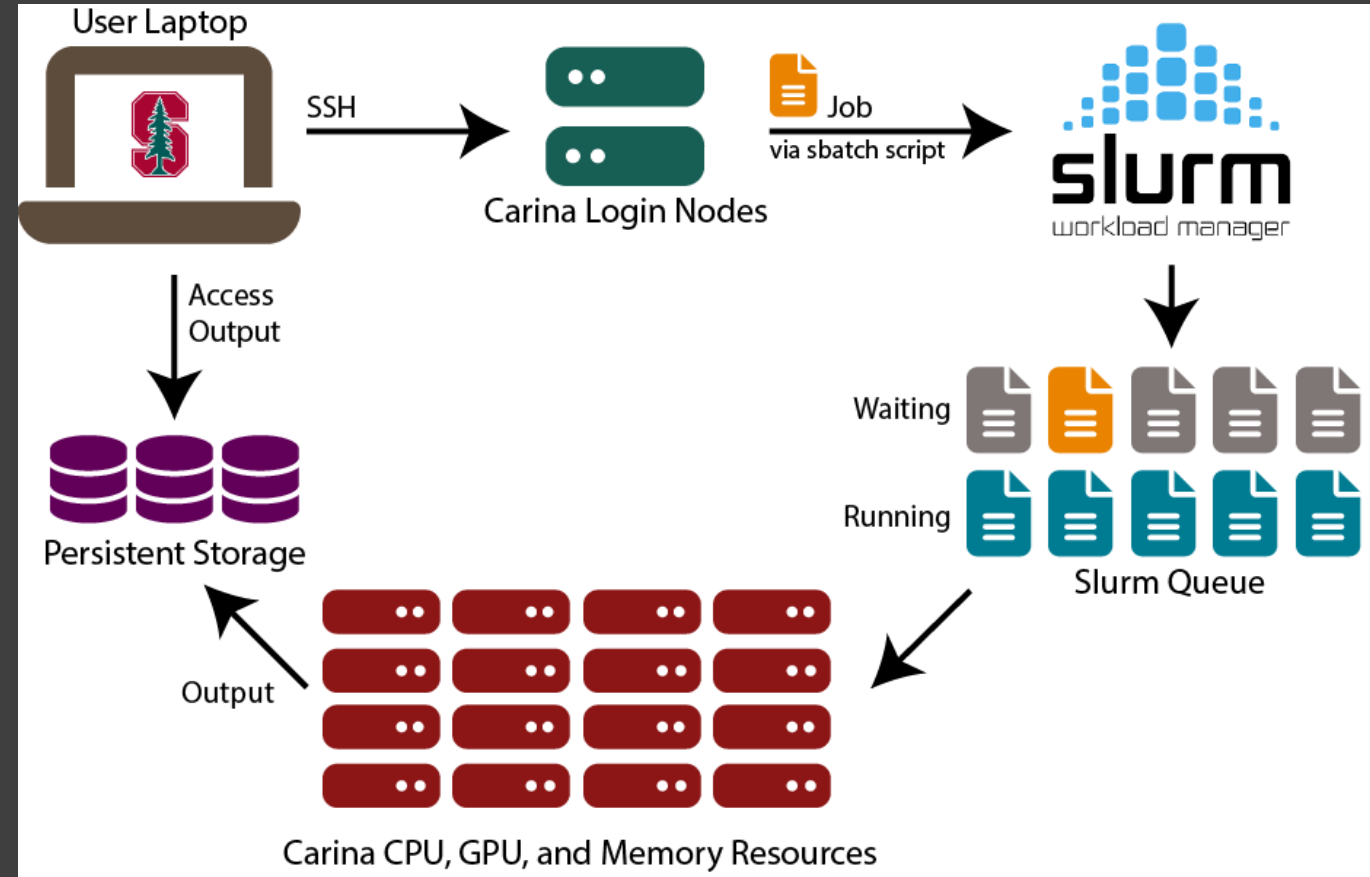
What is Slurm ?

- Slurm = Simple Linux Utility for Resource Management
- An open source system
 - 管理工作：資料分配、排程、執行



Roles in Slurm

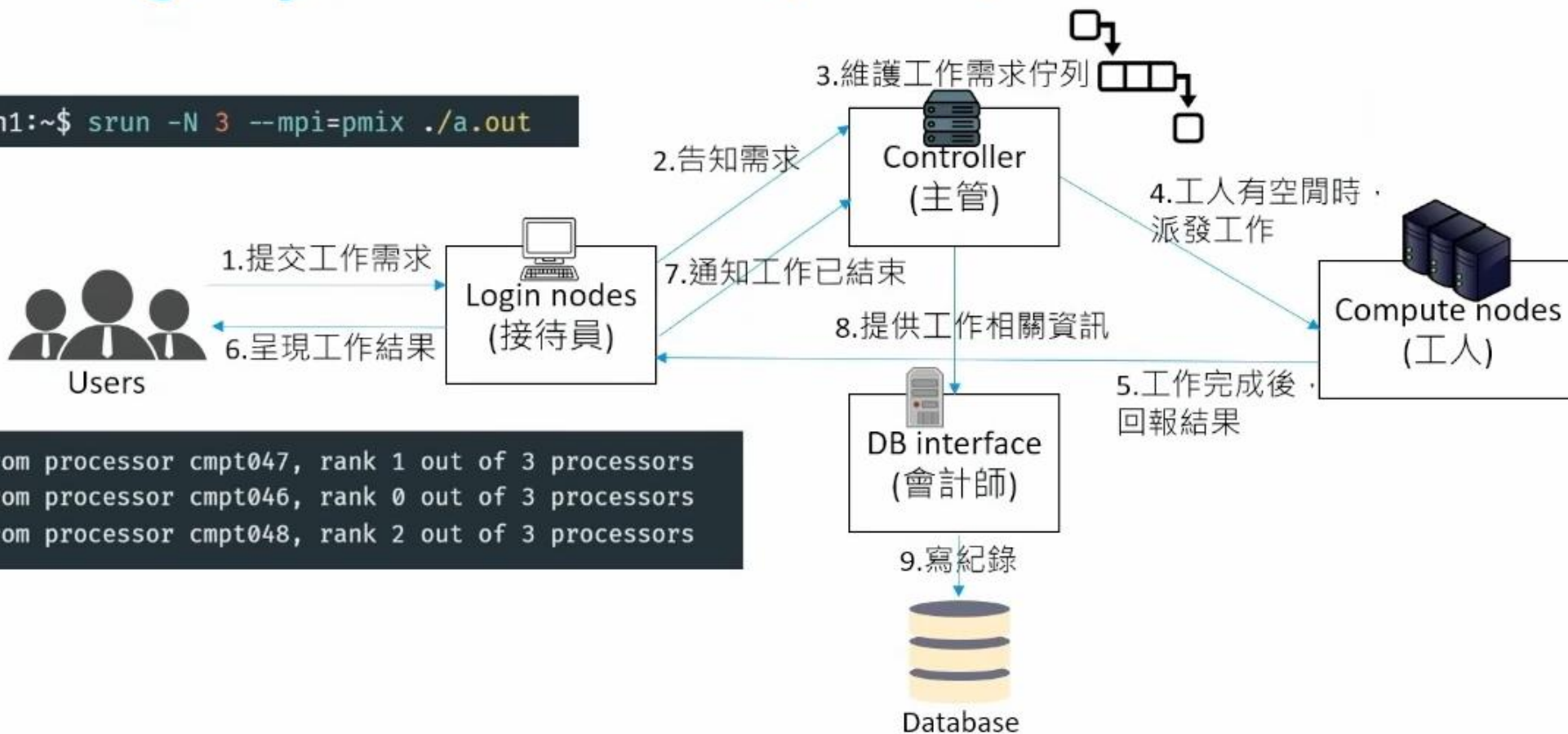
- Login nodes = 接待員/客服
- Controller = 主管
- Compute nodes = 工人
- DB interface = 會計師



Running a job in Slurm (srun)

Running a job in Slurm (srun)

```
bjhuang@login1:~$ srun -N 3 --mpi=pmix ./a.out
```



```
Hello world from processor cmpt047, rank 1 out of 3 processors  
Hello world from processor cmpt046, rank 0 out of 3 processors  
Hello world from processor cmpt048, rank 2 out of 3 processors
```

Kubernetes

What is Kubernetes ?

- Kubernetes (K8s) = Containers Orchestration Tool
- An open source system
 - 管理容器：啟動、運算資源、排程、自動部署
 - 大公司架設服務很愛用

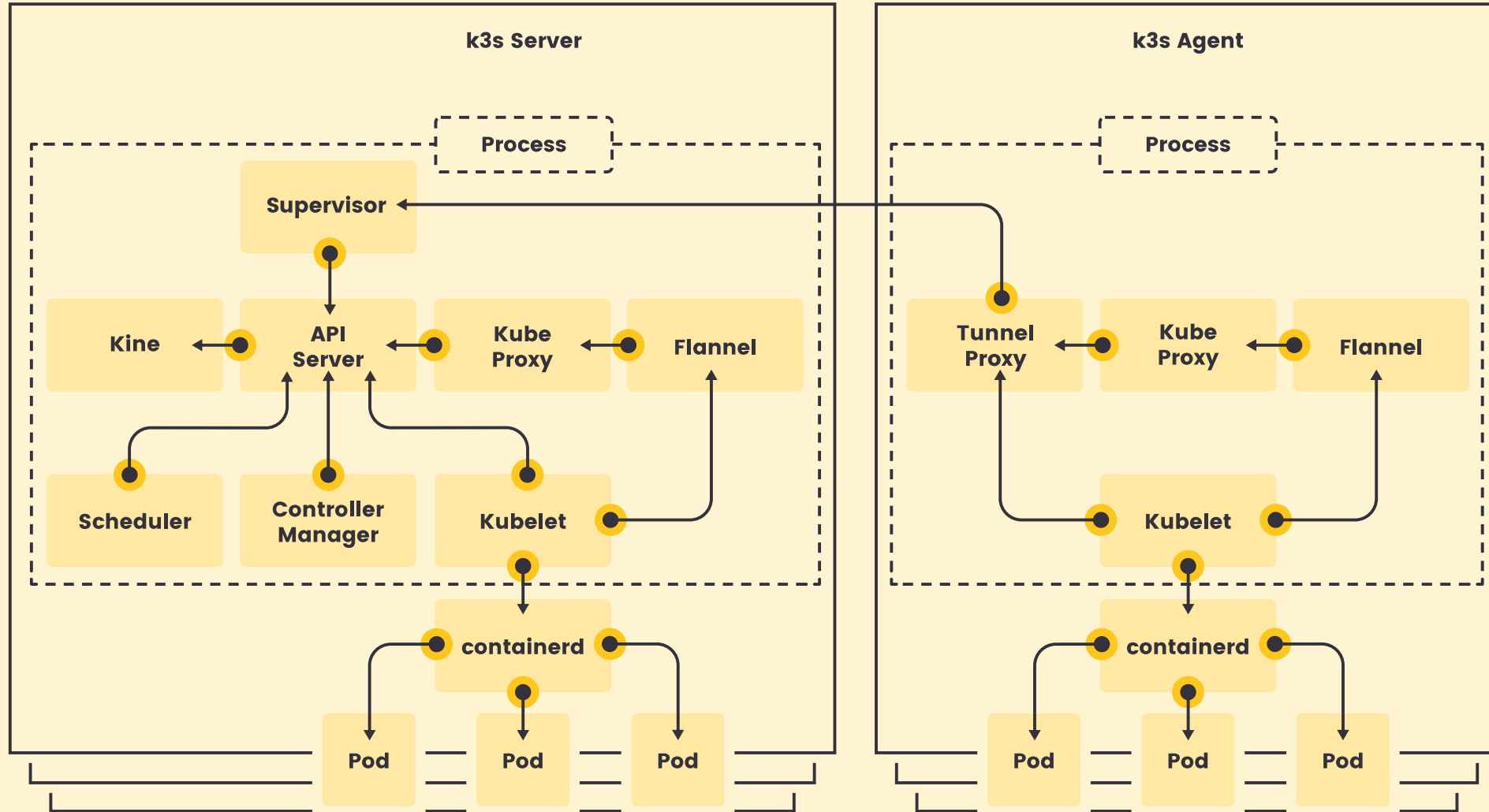


Kubernetes Distribution

- 本來是用 kind，但硬體支援度有極限，改成使用 K3s
- 建立在 Kubernetes 標準安裝之上的一個輕量發行版
- 由 Rancher Labs 開發，通過 CNCF 認證。



K3s



Background

Motivation

- Kubernetes 很擅長資源編排與彈性伸縮，但對 HPC / AI 訓練工作負載的語意支援有限。
 - Slurm 很擅長批次排程、資源配額與叢集治理，但傳統部署通常偏靜態，對雲端環境的節點生命週期不夠友善。
- ⇒ 打造一個讓多位使用者共用的批次 AI 工作平台



Benefits

- 提升資源利用率
 - Slurm 可以透過將工作排程到 K8s 叢集中的可用資源
- 簡化的工作負載管理
 - 一律透過 Slurm 介面，用於管理 ML 工作負載
- 提升可擴展性
 - Slurm 可以透過將機器學習工作負載排程到多個 K8s 叢集來擴展

Slurm on Kubernetes

- Slurm 決定誰用資源、什麼時候用、用多少
 - 計算資源排程問題
- Kubernetes 決定資源怎麼存在、怎麼啟動、怎麼管理
 - 動態提供 Slurm 需要的運算節點
 - 當硬體調整的時候不用重新調整 Slurm 設定
 - 依賴版本管理、網路設定

Our Feature !



- 擴容、擴縮、低功耗、節點壞掉重啟
- 客製化排程演算法、運算資源分配
- 支援多種 Job
- 近年來企業很重視安全，可以裝個龍蝦



Reliability



Performance



Lifecycle Unity



Observability



02. System Architecture

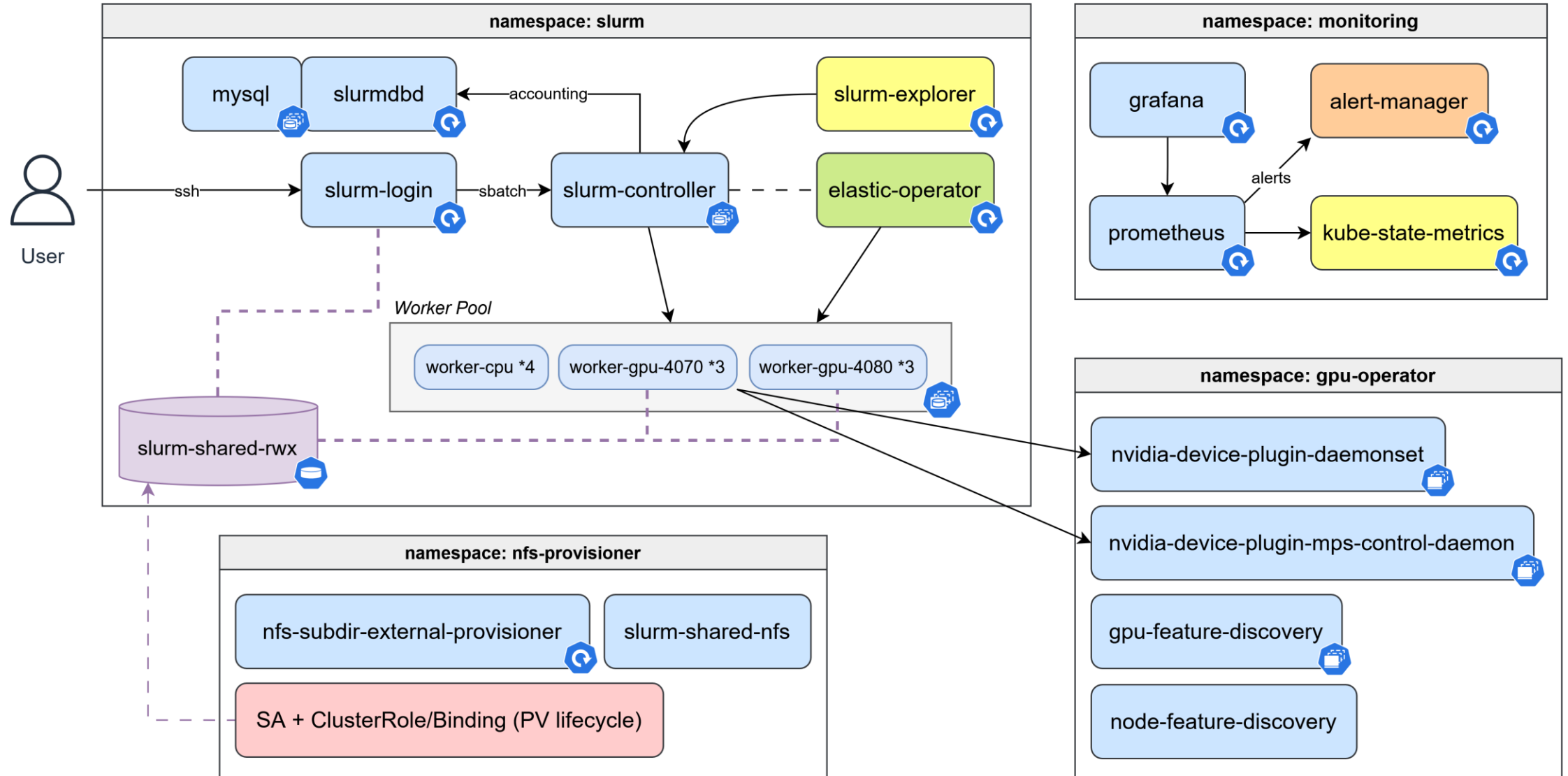
系統設計與架構

Development Environment



- Ubuntu 24.04 + K3s v1.34 + RTX 4070 (lab)
- Operator : Python 3.11-slim + kubectl CLI
- Nvidia GPU Operator

K8s (K3s) Cluster



Slurm GRES

- Slurm 的通用資源管理機制，讓 Slurm 能夠追蹤和排程 CPU/Memory 以外的資源 (e.g. GPU)
- 知道每個 node 上有哪些 GPU (型號、數量)
- 讓使用者提交 job 時指定需要的 GPU 類型和數量
- 確保 GPU 資源不會被過度分配

GPU Resources

Multi-Process Service (MPS)

- 在 GPU 上創建多個 context，將 GPU 的資源分配給多個 process
- 每個進程擁有一個獨立的 context，可以獨立執行其計算任務
- 提高多容器共享 GPU 的 QoS
- 不管 VRAM、記憶體頻寬、L2 Cache、編解碼器、Copy Engines
 - export `CUDA_MPS_PINNED_DEVICE_MEM_LIMIT=0,3072` (slot_0 3072 MB)

NVIDIA GPU MPS



請求 + constraint	RTX 4070 分配 SM	RTX 4080 分配 SM	適合工作
<code>--gres=mps:50 --constraint=gpu-rtx4070</code>	~24 SM (50%)	—	中型推論 (7B LLM serving)
<code>--gres=mps:25 --constraint=gpu-rtx4070</code>	~12 SM (25%)	—	小型推論 (image classifier)
<code>--gres=gpu:rtx4070:1</code>	48 SM (整卡)	—	訓練 (≤12 GB VRAM)
<code>--gres=gpu:rtx4080:1</code>	—	76 SM (整卡)	訓練 (≤16 GB VRAM)
<code>-N 2 --gres=gpu:1</code>	48 SM	76 SM	2-GPU DDP

Job Migration

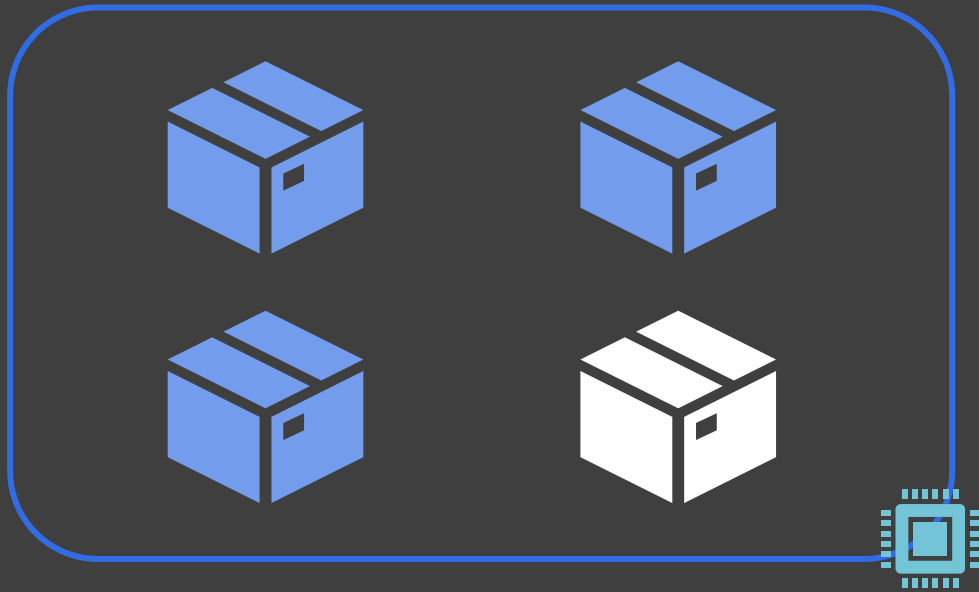
Job A



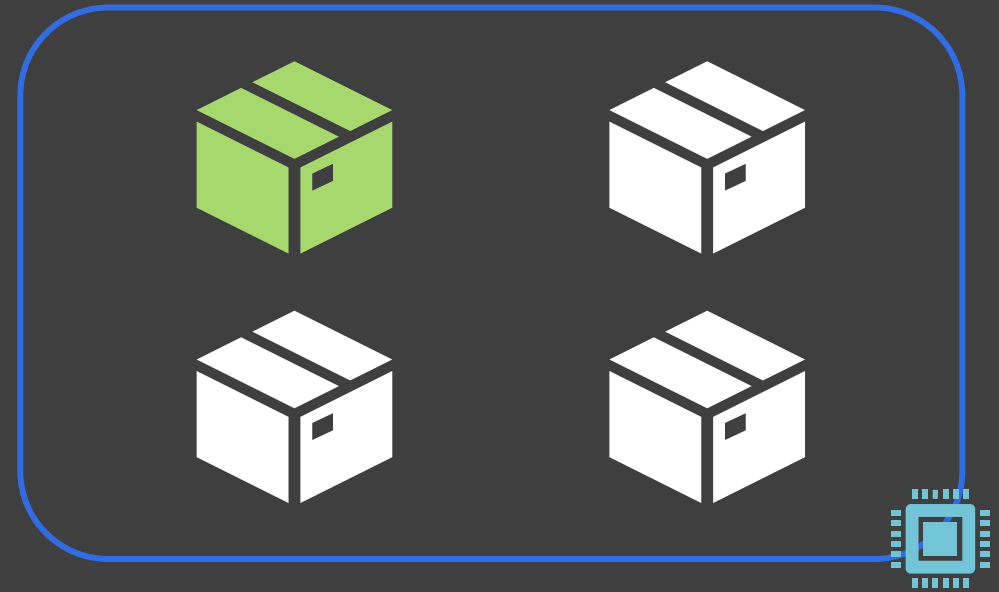
Job B



Job C

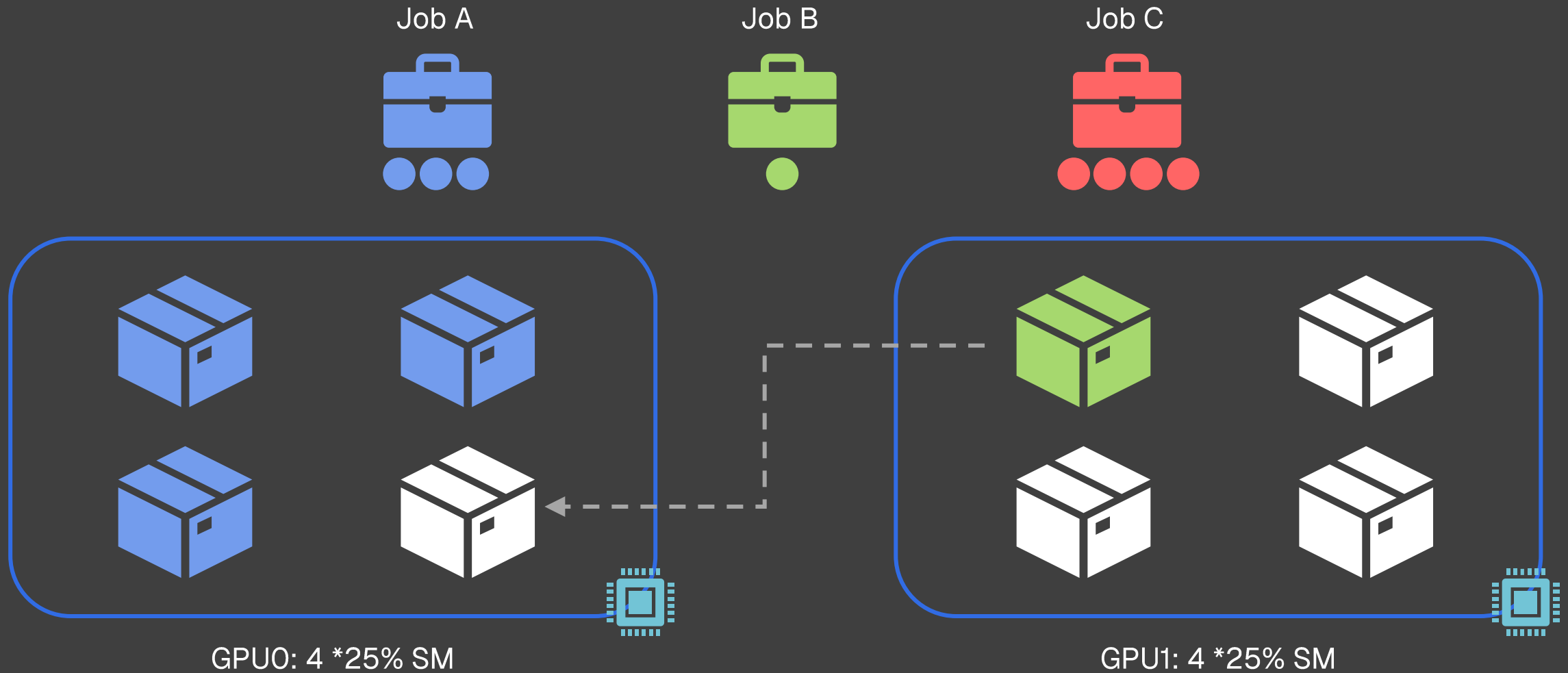


GPU0: 4 * 25% SM



GPU1: 4 * 25% SM

Job Migration



Job Migration

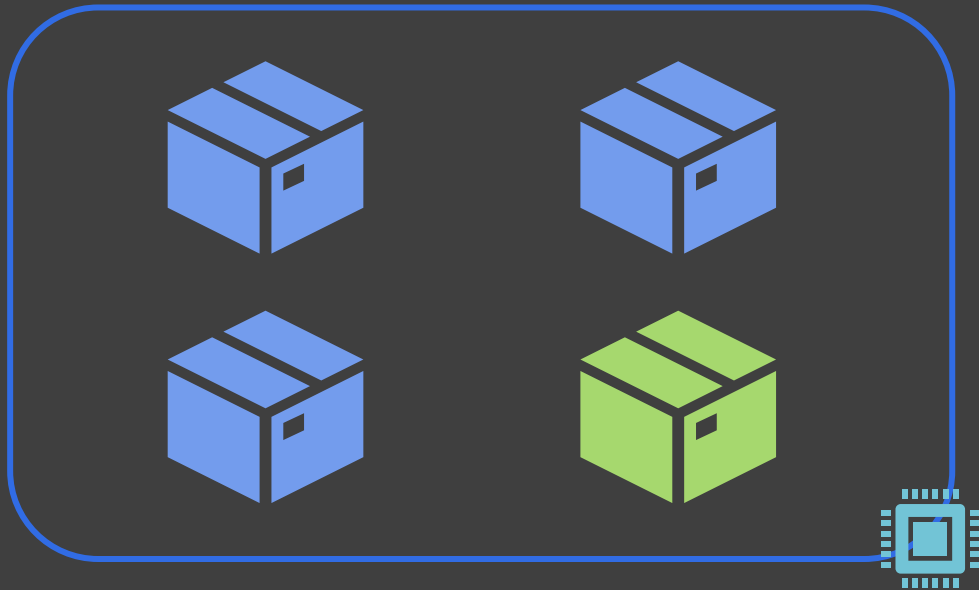
Job A



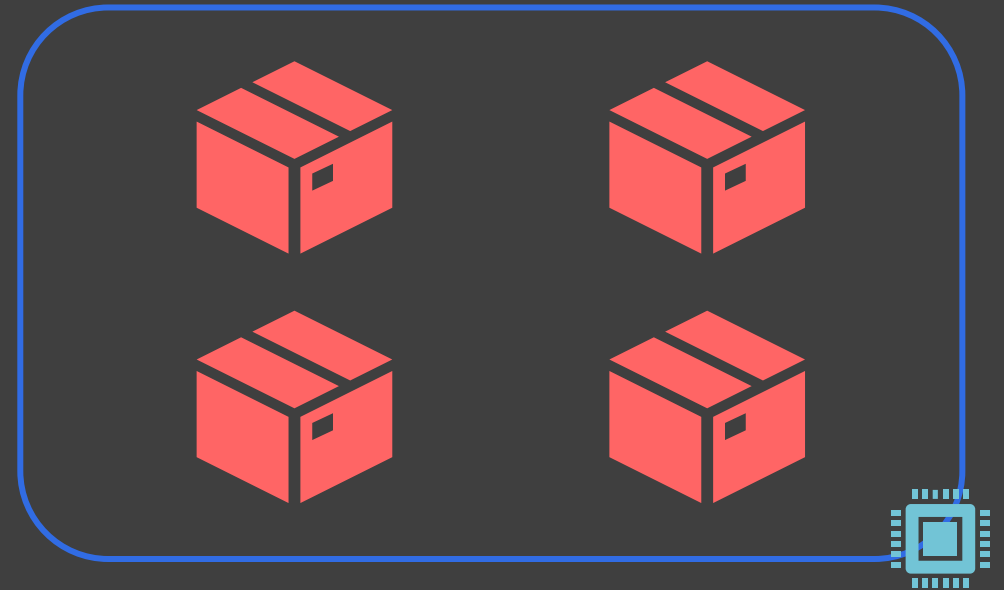
Job B



Job C



GPU0: 4 * 25% SM



GPU1: 4 * 25% SM

Job Scheduling / Migration

Slurm Scheduling Strategy (1/2)

- Bin-packing：先塞滿第一張卡，再用第二張卡
- 預設情況下，Slurm 為了散熱和負載平衡，傾向於把 Job 分散到不同的節點。

```
slurm:  
  config: |  
    SelectType=select/cons_tres  
    # 重點：CR_Core_Memory 配合 CR_Pack_Nodes  
    # 這會讓 Slurm 優先把任務填入同一個資源塊  
    SelectTypeParameters=CR_Core_Memory,CR_Pack_Nodes
```

Slurm Scheduling Strategy (2/2)

- Preemption：需要定義不同的 Partition 優先級
 - HighPrio Partition: 給 Job C (重要大任務)，優先級 10。
 - LowPrio Partition: 給 Job B (小任務)，優先級 1。
- 當 Job C 提交到 HighPrio 時，Slurm 會自動把 GPU1 上的 Job B 踢掉，讓 Job C 執行
- 等到 GPU0 或 GPU1 有空位時，Job B 會自動重新啟動

```
slurm:
  config: |
    PreemptMode=REQUEUE           # 或者用 SUSPEND (如果 Job 支援)
    PreemptType=preempt/partition_prio
    JobRequeue=1
```

Scheduling Strategy Design

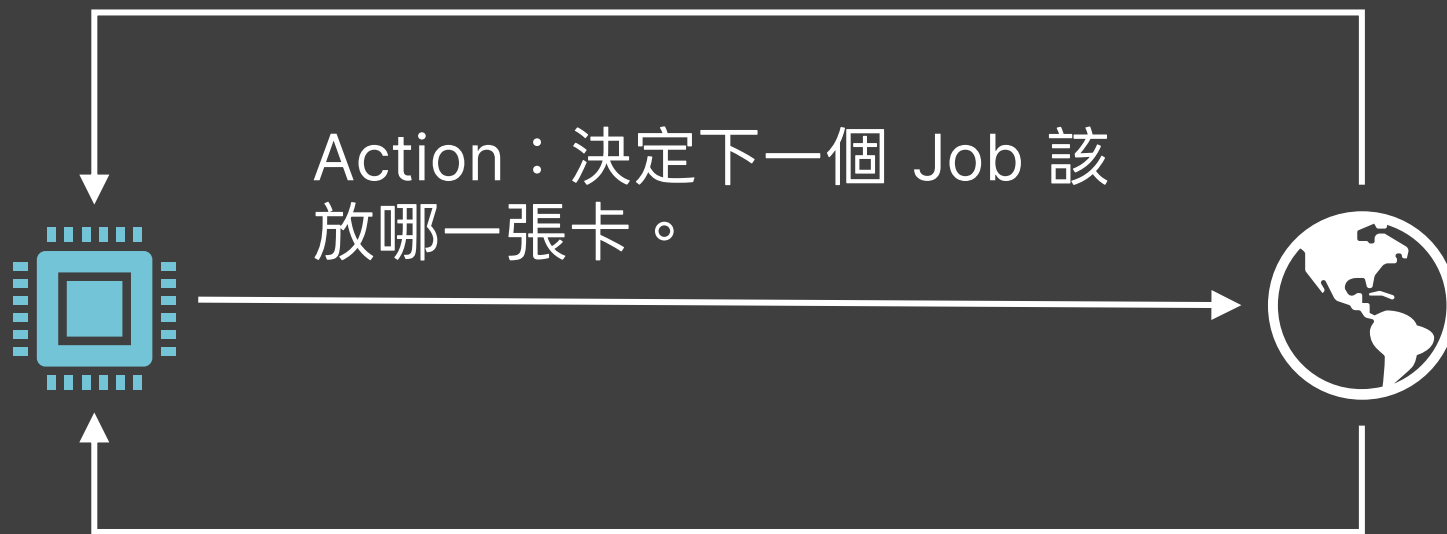
- 啟發式排程：改 Slurm 設定即可，快速有效
- 數學式排程：尋找最佳解
 - Analytical modeling + optimization
 - 深度強化學習 (DRL)

Analytical modeling + Optimization

- Score Function：給每個 (job, placement) 算分，挑最高的
- 係數 $\alpha, \beta, \gamma, \delta, \varepsilon$ 怎麼定？
 - 線上學習：把 $\alpha - \varepsilon$ 當可學習的權重，用 UCB1 等演算法線上更新。
 - 深度強化學習 DRL

Deep Reinforcement Learning (DRL) ***

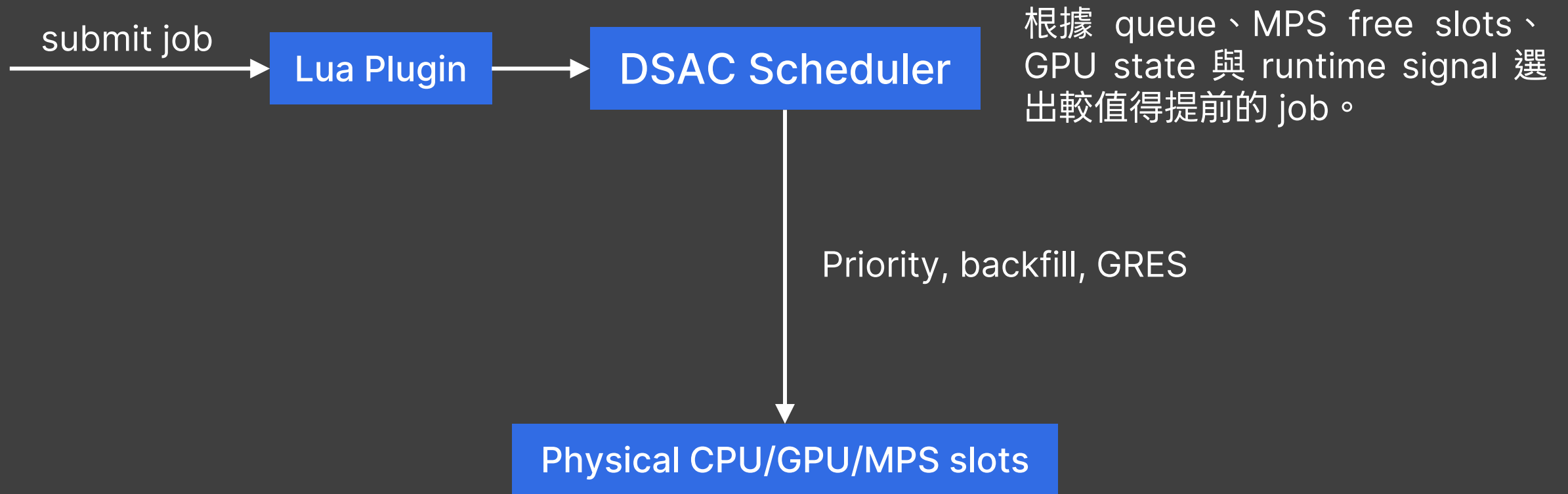
Reward：如果 Job 跑得快且資源沒浪費，給正分；如果發生 OOM 或排隊太久，給負分。



State：目前叢集所有 GPU 的剩餘資源、等待隊列中的 Job 屬性。

DQN < PPO < DDPG < SAC < DSAC-T

Scheduling Workflow



Placement Contract

Contract 欄位	來源	執行者	作用
<code>partition</code>	使用者指定或 submit helper 補齊	Slurm + Kelpflux operator	決定 CPU / GPU worker pool，例如 <code>cpu</code> 、 <code>gpu-rtx4070</code>
<code>constraint</code> / <code>features</code>	使用者指定	Slurm node feature matching	約束 VRAM tier、GPU 型號或節點特徵
<code>gres/gpu</code>	<code>--gres</code> / <code>tres_per_node</code>	Slurm GRES + NVIDIA device plugin	配置 GPU 類型與數量
<code>gres/mps</code>	<code>--gres</code> / <code>tres_per_node</code>	Slurm GRES + NVIDIA MPS control daemon	配置單 GPU 上的 MPS slot
<code>memory</code>	使用者指定或 helper 估算	Slurm cgroup / Kubernetes pod resources	避免 job 在 placement 後因 memory 不足失敗
<code>qos</code> / <code>priority</code>	使用者、helper、score、DSAC	Slurm priority queue	決定 pending jobs 的排序與 backfill 機會
worker pool size	Slurm pending state	Kelpflux operator	依 pending jobs 擴縮 Kubernetes worker StatefulSet

DSAC in Placement

- 模型輸出的 action 會被解碼成 $(job_i, node_j, gpu_k)$, action mask 只允許選擇 MPS free slots 足夠的 placement 。
- 這個設計讓 live cluster 可以先安全使用 RL 影響排序，同時保留完整 placement-aware state/action 資料。

DSAC I/O

(Input)

Job

GPU/Node

queue jobs

GPU/MPS resource snapshot

feasibility mask

(Output)

job_i

rem

$node_j$

gpu_k

選 pending queue 中第 job_i 個 job

放到第 $node_j$ 個 node

使用第 gpu_k 個 GPU/MPS slot

DSAC in Placement

DSAC 輸出	Submit-time live path	Hard placement controller
<code>job_i</code>	若選中的 job 是正在 submit 的 job，回傳 positive <code>priority_boost</code>	對 held pending queue 中被選到的 job 執行 placement
<code>node_j</code>	記錄為 placement intent，用於 metrics、Grafana 與訓練分析	映射到可用 Slurm GPU worker node，寫入 <code>ReqNodeList=<node></code>
<code>gpu_k</code>	記錄為 placement intent，用於觀察模型偏好的 GPU slot	目前 live worker 是 1 GPU / node，因此 <code>gpu_k=0</code> ；多 GPU node 需要對應 GPU/MPS slot topology
<code>priority_boost</code>	加到 <code>job_desc.priority</code> ，讓 Slurm 更早考慮該 job	不使用；controller 透過 hold-release 控制何時進入 Slurm placement
<code>abstain</code> / no-op	guardrail 觸發時不 boost，回到 score + Slurm	不更新 job，保持 held/pending，等待下一輪或人工處理

Observability

Observability



雲端服務崛起



錯誤排查難度上升

Observability Aspects

Metrics



負責監控系統有什麼狀況，當要發生服務故障前可以透過設定閾值搭配告警提早得知。

Logs

當問題發生時，可以用來查看故障時正在執行哪些服務，以及產生的錯誤資訊。

Traces



在連續的時間維度上，透過 Trace 以及 Span 關聯，把空間給排列展示出來，並且有 Trace-Context 規範，能夠直觀的看到請求在分散式系統中經過所有服務的紀錄。

GPU Monitor

GPU profile — acane (gpu=All)

SM Utilisation (%) ⓘ



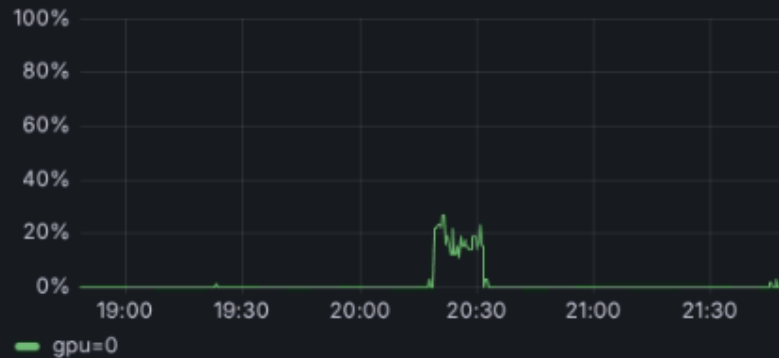
Name	Mean	Max
gpu=0 NVIDIA GeForce RTX 4070	3.16%	58%

VRAM Used (MiB) ⓘ



Name	Mean	Max
gpu=0 used	510 MB	4.39 GB
gpu=0 free	11.4 GB	11.7 GB

Memory Copy Util (%) ⓘ



Power Draw (W) ⓘ



Temperature (°C) ⓘ



DSAC Scheduler Monitor

